

Criptanálise Diferencial em Cifras de Fluxo

*Análise Matemática e Implementação de Ataques
sobre LCG em $\mathbb{F}_{2^e}$ com Função Filtro de Baixo Grau*

Baseado em:

Eli Biham & Orr Dunkelman

*“Differential Cryptanalysis in Stream Ciphers”
(Biham & Dunkelman, 2007)*

Matheus Sousa

Departamento de Eletrônica e Sistemas — UFPE

9 de março de 2026

Resumo. Este relatório apresenta a fundamentação matemática completa e a análise da implementação dos ataques de criptanálise diferencial descritos por Biham e Dunkelman aplicados a uma cifra de fluxo sintética modelada como um *Gerador Congruencial Linear* (LCG) sobre o corpo finito $\mathbb{F}_{2^e}$, combinado com uma função filtro booleana de grau algébrico baixo. São provados, com álgebra booleana e linear, todos os invariantes explorados pelos seis módulos de ataque do projeto: recuperação de chave por sistema linear, correlação diferencial, ataque distinguidor, diferencial de segunda ordem, múltiplas características e aceleração da busca exaustiva. Por fim, analisa-se a extensão para funções filtro de grau algébrico arbitrário d , explicitando o custo 2^d do diferencial de ordem d e as condições de inviabilidade computacional.

Contents

1	Introdução e Modelo do Sistema	2
1.1	Contextualização	2
1.2	Representação em $\mathbb{F}_{2^e}$	2
1.3	Configurações de Teste	2
2	Fundamentação: Linearidade do LCG e Propagação de Diferenças	3
3	Módulo 1: batch_attack + recover_key	4
3.1	Estratégia do Ataque	4
3.2	Prova: Linearização da Diferença de Saída	4
3.3	Prova: Validade no Tempo (Propagação pelo LCG)	4
3.4	Recuperação do Estado Inicial: Sistema Linear	5
3.5	Correspondência com o Código	5
4	Módulo 2: differential_correlation	6
4.1	Correlação Passiva: $\phi(z_t, s_{20})$	6
4.2	Correlação Diferencial: $\phi(\Delta z_t, s_{10})$	6
4.3	Coeficiente de Matthews	6
5	Módulo 3: distinguish_attack	7
5.1	Condição de Distinguibilidade	7
5.2	Fórmula Geral da Diferença de Saída	7
6	Módulo 4: higher_order_differential	8
6.1	Diferenciais de Ordem k : Definição	8
6.2	Teorema Central: Grau Algébrico e Colapso do Diferencial	8
6.3	Aplicação à Função Filtro f (Grau 2)	9
7	Módulo 5: multi_difference_attack	9
7.1	Recuperação de Bits por Características Complementares	9
7.2	Conjunto de Diferenças Utilizadas	10
8	Módulo 6: exhaustive_speedup	10
8.1	Propriedade de Simetria dos Pares	10
8.2	Verificação Experimental	10
9	Módulo 7: higher_order_stress e higher_order_scaling	11
9.1	Custo Assintótico do Ataque de Ordem k	11
9.2	Tabela de Viabilidade	11
9.3	Análise do higher_order_stress	11
9.4	Análise do higher_order_scaling	12

10 Extensão para Funções Filtro de Grau Maior	12
10.1 Generalização do Ataque de Recuperação de Chave	12
10.2 Generalização da Recuperação de Estado	13
10.3 Limiares de Segurança	13
11 Síntese dos Resultados Experimentais	14
12 Conclusões	14
A Identidades de Álgebra Booleana Utilizadas	15
B Propriedades de $\mathbb{F}_{2^e}$ Utilizadas	15
C Referência: Paper Original	15

1 Introdução e Modelo do Sistema

1.1 Contextualização

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman definem no §3.1 as três categorias de características diferenciais para cifras síncronas:

$$(\Delta_{\text{key}}, \Delta_{IV}) \xrightarrow{\text{INIT}} \Delta S, \quad \Delta S \xrightarrow{\text{UPDATE}} \Delta S, \quad \Delta S \xrightarrow{\text{OUTPUT}} \Delta KS.$$

O projeto implementa exatamente este modelo, explorando a composição $\Delta_0 \xrightarrow{\text{INIT+OUTPUT}} \Delta KS$ com probabilidade 1 (característica perfeita).

A cifra de fluxo estudada neste projeto é formada por dois componentes:

- (i) **Gerador de estado interno (UPDATE):** um Gerador Congruencial Linear (LCG) definido sobre o corpo finito $\mathbb{F}_{2^e}$:

$$x_{t+1} = a \cdot x_t + c, \quad a, c, x_0 \in \mathbb{F}_{2^e},$$

onde $\mathbb{F}_{2^e} \cong \mathbb{F}_2[x]/\langle h(x) \rangle$, com $h(x)$ polinômio primitivo de grau e sobre $\mathbb{F}_2$.

- (ii) **Função filtro (OUTPUT):** uma função booleana $f : \mathbb{F}_{2^e} \rightarrow \mathbb{F}_2$ que extrai 1 bit de keystream por clock:

$$z_t = f(x_t) = s_{20}(x_t) \oplus (s_0(x_t) \wedge s_{10}(x_t)),$$

onde $s_k(x)$ denota o coeficiente do monômio x^k na representação polinomial de x em $\mathbb{F}_{2^e}$.

1.2 Representação em $\mathbb{F}_{2^e}$

Cada estado $x_t \in \mathbb{F}_{2^e}$ é representado como um vetor de e bits:

$$x_t = \sum_{k=0}^{e-1} s_k(x_t) \xi^k, \quad s_k(x_t) \in \mathbb{F}_2,$$

onde ξ é raiz de h . A multiplicação por a em $\mathbb{F}_{2^e}$ é uma transformação $\mathbb{F}_2$ -linear, fato fundamental para a validade de todos os ataques.

1.3 Configurações de Teste

O arquivo `unified_config.json` fornece 8 instâncias cobrindo diferentes tamanhos de campo:

Table 1: Configurações de campo do projeto

Config	Campo	$h(x)$	$a(x)$
1	$\mathbb{F}_{2^{31}}$	$x^{31} + x^{13} + x^8 + x^3 + 1$	$x^4 + 1$
2	$\mathbb{F}_{2^{64}}$	$x^{64} + x^{11} + x^2 + x + 1$	$x^4 + 1$
3	$\mathbb{F}_{2^{64}}$	$x^{64} + x^{42} + x^{33} + x^{22} + 1$	$x^5 + 1$
4	$\mathbb{F}_{2^{128}}$	$x^{128} + x^{79} + x^{51} + x^{29} + 1$	$x^5 + 1$
5	$\mathbb{F}_{2^{256}}$	$x^{256} + x^{213} + x^{141} + x^{51} + 1$	$x^6 + 1$
6	$\mathbb{F}_{2^{271}}$	$x^{271} + x^{188} + x^{121} + x^{93} + 1$	(pentanômio)
7	$\mathbb{F}_{2^{291}}$	$x^{291} + x^{240} + x^{202} + x^{178} + 1$	(pentanômio)
8	$\mathbb{F}_{2^{451}}$	$x^{451} + x^{402} + x^{292} + x^9 + 1$	(pentanômio)

2 Fundamentação: Linearidade do LCG e Propagação de Diferenças

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4, parágrafo inicial): *“It is easy to see that given two (key, IV) pairs ... the difference in the internal state is known for any given number of clocks. This allows to estimate the output difference of the combining function.”* A prova formal abaixo estabelece este resultado para o nosso LCG.

Teorema 2.1 (Propagação Linear de Diferenças no LCG). *Sejam $x_0, x'_0 \in \mathbb{F}_{2^e}$ dois estados iniciais com diferença $\Delta_0 = x_0 \oplus x'_0$. Após t clocks do LCG $x \mapsto ax + c$, a diferença entre os estados é:*

$$\Delta_t := x_t \oplus x'_t = a^t \cdot \Delta_0.$$

Proof. Por indução em t . **Base:** $t = 0$, $\Delta_0 = x_0 \oplus x'_0$. **Passo:** supondo $x_t \oplus x'_t = a^t \Delta_0$,

$$\begin{aligned}
 x_{t+1} \oplus x'_{t+1} &= (ax_t + c) \oplus (ax'_t + c) \\
 &= a(x_t \oplus x'_t) \oplus \underbrace{(c \oplus c)}_0 \quad (\text{linearidade de } \mathbb{F}_{2^e}) \\
 &= a \cdot a^t \Delta_0 = a^{t+1} \Delta_0.
 \end{aligned}$$

□

Observação 2.2. O termo c cancela exatamente porque a subtração em $\mathbb{F}_2$ coincide com a adição ($x \oplus x = 0$). Isso é a causa central da vulnerabilidade: a diferença evolui *deterministicamente* e é completamente previsível pelo atacante a partir do momento em que Δ_0 é fixado.

3 Módulo 1: batch_attack + recover_key

3.1 Estratégia do Ataque

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4.1.3, *Differential Key Recovery Attack on Toyocrypt*): “Given two key streams produced by two keys whose initial state difference is $\Delta S = s_0$, it is possible to reconstruct most of the internal state.” No projeto, a injeção de $\Delta_0 = 1$ (i.e., $s^0 = x^0$, o monômio constante em $\mathbb{F}_{2^e}$) serve ao mesmo propósito.

O atacante fixa $\Delta_0 = 1 \in \mathbb{F}_{2^e}$ (bit 0 invertido) e observa, para cada t , o par (z_t, z'_t) onde $z_t = f(x_t)$ e $z'_t = f(x_t \oplus \Delta_0)$ (sonda local, não requer dois fluxos simultâneos).

3.2 Prova: Linearização da Diferença de Saída

Teorema 3.1 (Extração de bit de estado via diferença de saída). *Seja $f(x) = s_{20}(x) \oplus (s_0(x) \wedge s_{10}(x))$ e $\Delta_0 = 1$ (monômio x^0). Para qualquer estado $x_t \in \mathbb{F}_{2^e}$:*

$$\Delta z_t := f(x_t) \oplus f(x_t \oplus \Delta_0) = s_{10}(x_t).$$

Proof. Seja $p = x_t$ e $p' = x_t \oplus 1$. Então $s_k(p') = s_k(p)$ para todo $k \neq 0$, e $s_0(p') = 1 \oplus s_0(p)$. Portanto:

$$\begin{aligned} f(p) &= s_{20}(p) \oplus (s_0(p) \wedge s_{10}(p)), \\ f(p') &= s_{20}(p) \oplus ((s_0(p) \oplus 1) \wedge s_{10}(p)). \end{aligned}$$

Calculando $f(p) \oplus f(p')$:

$$\begin{aligned} \Delta z &= [s_{20} \oplus (s_0 \wedge s_{10})] \oplus [s_{20} \oplus ((s_0 \oplus 1) \wedge s_{10})] \\ &= (s_0 \wedge s_{10}) \oplus [(s_0 \oplus 1) \wedge s_{10}] \\ &= s_{10} \wedge [s_0 \oplus (s_0 \oplus 1)] \quad (\text{distributividade de AND sobre XOR}) \\ &= s_{10} \wedge 1 = s_{10}. \end{aligned} \quad \square$$

Observação 3.2. A prova usa a identidade distributiva da álgebra de Boole: $(A \oplus B) \wedge C = (A \wedge C) \oplus (B \wedge C)$, e o fato de que $s_0 \oplus (s_0 \oplus 1) = 1$ em $\mathbb{F}_2$.

3.3 Prova: Validade no Tempo (Propagação pelo LCG)

Teorema 3.3 (Extração temporal de s_{10}). *A sonda local $f(x_t) \oplus f(x_t \oplus \Delta_0)$ extrai $s_{10}(x_t)$ com probabilidade 1 para todo $t \geq 0$.*

Proof. Em cada clock o atacante computa $x_t \oplus \Delta_0$ diretamente, sem precisar propagar Δ_0 ; basta inverter o bit 0 do estado corrente. Pelo Teorema 3.1, o resultado $\Delta z_t = s_{10}(x_t)$ vale para qualquer x_t , em particular para $x_t = a^t x_0 + c_t$ onde $c_t = \sum_{i=0}^{t-1} a^i c$ é a soma de deslocamento acumulada. $\square$

3.4 Recuperação do Estado Inicial: Sistema Linear

Referência ao Paper (Biham & Dunkelman)

O método é o análogo algébrico da recuperação descrita por Biham & Dunkelman (§4.1.3) para a Toyocrypt: identificados e bits de estado ao longo do tempo, monta-se um sistema linear invertível.

Após observar $e = \deg(h)$ clocks, o atacante dispõe do vetor $B = (\Delta z_0, \dots, \Delta z_{e-1})^T = (s_{10}(x_0), \dots, s_{10}(x_{e-1}))^T \in \mathbb{F}_2^e$.

Escrevemos $x_t = a^t x_0 + C_t$ onde $C_t = \sum_{i=0}^{t-1} a^i c$. O bit $s_{10}(x_t)$ é a projeção linear $\langle e_{10}, x_t \rangle$ no sentido de $\mathbb{F}_2^e$, portanto:

$$B_t = s_{10}(a^t x_0 + C_t) = s_{10}(a^t x_0) \oplus s_{10}(C_t).$$

Definindo $Y_t := B_t \oplus s_{10}(C_t)$, temos $Y_t = s_{10}(a^t x_0)$. O mapeamento $x_0 \mapsto s_{10}(a^t x_0)$ é $\mathbb{F}_2$ -linear; sua matriz $M \in \mathbb{F}_2^{e \times e}$ tem entradas:

$$M_{t,j} = s_{10}(a^t \cdot \xi^j), \quad t, j \in \{0, \dots, e-1\},$$

onde ξ^j é o j -ésimo elemento da base canônica de $\mathbb{F}_{2^e}$.

Teorema 3.4 (Invertibilidade da Matriz de Recuperação). *Se a é um elemento primitivo de $\mathbb{F}_{2^e}$ (ou, mais geralmente, se o sequenciador $x \mapsto ax + c$ tem período $2^e - 1$), então M é invertível sobre $\mathbb{F}_2$ e o estado x_0 é unicamente recuperado por:*

$$\vec{S}_0 = M^{-1}Y,$$

onde $\vec{S}_0 \in \mathbb{F}_2^e$ é o vetor de bits do estado inicial x_0 .

Prova (esboço). As e funções lineares $\ell_t(x_0) = s_{10}(a^t x_0)$ são $\mathbb{F}_2$ -linearmente independentes porque o mapa $t \mapsto a^t$ percorre uma órbita de comprimento $2^e - 1$ no grupo multiplicativo $\mathbb{F}_{2^e}^*$, e a projeção s_{10} é uma forma linear não-nula em $\mathbb{F}_{2^e}$. A linearidade independente das linhas de M garante $\det(M) \neq 0$ sobre $\mathbb{F}_2$. $\square$

3.5 Correspondência com o Código

Em `recover_key.cpp`, o laço constrói exatamente M e Y :

- `Linha M[i][j] = IsOne(coeff(rep(a_pow * xj), target_bit))` $\leftrightarrow M_{i,j} = s_{10}(a^i \cdot \xi^j)$.
- `Linha Y[i] = B_i ^ c_bit` $\leftrightarrow Y_i = B_i \oplus s_{10}(C_i)$.
- `inv(M_inv, M)` resolve M^{-1} em $\mathbb{F}_2^{e \times e}$ pela biblioteca NTL.

4 Módulo 2: differential_correlation

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4, segundo parágrafo): *“This fact can also be used to distinguish the output of the stream cipher under two related keys (or IVs) from two random strings. Moreover, in some cases, it can be used to identify information about the internal state.”* O coeficiente de correlação ϕ de Matthews quantifica precisamente o viés que Biham & Dunkelman descrevem informalmente.

4.1 Correlação Passiva: $\phi(z_t, s_{20})$

A função $f(x) = s_{20} \oplus (s_0 \wedge s_{10})$ possui um viés mensurável entre a saída z_t e o bit $s_{20}(x_t)$:

Proposição 4.1 (Viés passivo da saída). *Assumindo x_t uniformemente distribuído em $\mathbb{F}_{2^e}$, a correlação de Matthews entre $z_t = f(x_t)$ e $s_{20}(x_t)$ é:*

$$\phi(z_t, s_{20}(x_t)) > 0.$$

Proof. Calculamos a tabela de contingência. Como s_0, s_{10}, s_{20} são bits estatisticamente independentes e uniformes em $\{0, 1\}$, temos $\Pr[s_0 = 1] = \Pr[s_{10} = 1] = \Pr[s_{20} = 1] = 1/2$.

Para $\phi(z, s_{20}) \neq 0$, basta mostrar $P(z = s_{20}) \neq 1/2$:

$$\begin{aligned} P(z = s_{20}) &= P[s_{20} \oplus (s_0 \wedge s_{10}) = s_{20}] \\ &= P[s_0 \wedge s_{10} = 0] \\ &= 1 - P[s_0 = 1] \cdot P[s_{10} = 1] = 1 - \frac{1}{4} = \frac{3}{4}. \end{aligned}$$

Portanto $\phi > 0$ (correlação positiva). □

4.2 Correlação Diferencial: $\phi(\Delta z_t, s_{10})$

Teorema 4.2 (Correlação diferencial perfeita). *Seja $\Delta z_t = f(x_t) \oplus f(x_t \oplus 1)$. Então:*

$$\phi(\Delta z_t, s_{10}(x_t)) = 1.$$

Proof. Pelo Teorema 3.1, $\Delta z_t = s_{10}(x_t)$ com probabilidade 1, portanto a concordância é total: os eventos $\{\Delta z_t = 1\}$ e $\{s_{10}(x_t) = 1\}$ são idênticos. O coeficiente de Matthews de dois eventos idênticos é 1. □

4.3 Coeficiente de Matthews

O código usa o coeficiente ϕ de Matthews definido como:

$$\phi = \frac{n_{11}n_{00} - n_{10}n_{01}}{\sqrt{(n_{11} + n_{10})(n_{01} + n_{00})(n_{11} + n_{01})(n_{10} + n_{00})}},$$

onde n_{ij} conta os pares em que a variável $X = i$ e $Y = j$. Pelo Teorema 4.2, para o par $(\Delta z, s_{10})$: $n_{10} = n_{01} = 0$, logo $\phi = 1$.

5 Módulo 3: distinguish_attack

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4.1.1, *Distinguishing Attack on Toyocrypt*): “given two streams generated by related keys, we check whether the first output bit is the same and the second one differs. If this is the case, we output that the stream cipher in use is Toyocrypt.” O distinguidor do projeto é uma generalização determinística desta ideia: identifica os clocks em que a diferença interna Δ_t tem $\Delta_0 = \Delta_{10} = 0$, momento em que $\Delta z_t = \Delta_{20}$ com probabilidade 1.

5.1 Condição de Distinguiabilidade

Teorema 5.1 (Distinguidor Determinístico). *Seja $\Delta_t = a^t \Delta_0$ com $\Delta_0 = 1$. Para os clocks t tais que $s_0(\Delta_t) = s_{10}(\Delta_t) = 0$, a diferença de saída satisfaz:*

$$\Delta z_t = s_{20}(\Delta_t).$$

Portanto, o evento “par de streams com diferença inicial $\Delta_0 = 1$ ” é distinguível de dois streams aleatórios independentes com taxa de erro 0%.

Proof. Seja $\Delta = \Delta_t$ no instante t , e sejam $p = x_t$, $p' = x_t \oplus \Delta$. Por definição, $s_k(p') = s_k(p) \oplus s_k(\Delta)$ para todo k (linearidade da projeção).

Calculamos $f(p) \oplus f(p')$:

$$\begin{aligned} \Delta z &= [s_{20}(p) \oplus (s_0(p) \wedge s_{10}(p))] \\ &\quad \oplus [(s_{20}(p) \oplus \delta_{20}) \oplus ((s_0(p) \oplus \delta_0) \wedge (s_{10}(p) \oplus \delta_{10}))] \end{aligned}$$

onde $\delta_k = s_k(\Delta)$. Expandindo:

$$\Delta z = \delta_{20} \oplus [s_0 \wedge s_{10}] \oplus [(s_0 \oplus \delta_0) \wedge (s_{10} \oplus \delta_{10})].$$

Expandindo o produto booleano:

$$(s_0 \oplus \delta_0)(s_{10} \oplus \delta_{10}) = s_0 s_{10} \oplus s_0 \delta_{10} \oplus \delta_0 s_{10} \oplus \delta_0 \delta_{10}.$$

Portanto:

$$\Delta z = \delta_{20} \oplus s_0 \delta_{10} \oplus \delta_0 s_{10} \oplus \delta_0 \delta_{10}.$$

Sob a condição $\delta_0 = s_0(\Delta_t) = 0$ e $\delta_{10} = s_{10}(\Delta_t) = 0$:

$$\Delta z = \delta_{20} \oplus 0 \oplus 0 \oplus 0 = \delta_{20} = s_{20}(\Delta_t). \quad \square$$

5.2 Fórmula Geral da Diferença de Saída

Corolário 5.2 (Equação diferencial completa). *Para quaisquer $\delta_0, \delta_{10}, \delta_{20} \in \mathbb{F}_2$:*

$$\Delta z_t = \delta_{20} \oplus (s_0(x_t) \wedge \delta_{10}) \oplus (\delta_0 \wedge s_{10}(x_t)) \oplus (\delta_0 \wedge \delta_{10}).$$

Esta fórmula unificada é usada diretamente pelo `multi_difference_attack` (Seção 7) com diferentes valores de Δ_0 .

6 Módulo 4: higher_order_differential

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4, parágrafo sobre monômio de grau 3): “if there is a difference in only one bit which is combined through a monomial of degree 3, there is a probability of $1/4$ that there is a difference in the key streams in the corresponding bit.” O diferencial de ordem superior é exatamente a ferramenta para contornar esta probabilidade fracionária e recuperar informação determinística quando o grau algébrico é conhecido.

6.1 Diferenciais de Ordem k : Definição

Definição 6.1 (Diferencial de ordem k). Dada $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ e vetores $\alpha_1, \dots, \alpha_k \in \mathbb{F}_2^n$, o diferencial de ordem k de f no ponto x é:

$$D_{\alpha_1, \dots, \alpha_k}^k f(x) := \bigoplus_{S \subseteq [k]} f\left(x \oplus \bigoplus_{i \in S} \alpha_i\right),$$

onde a soma é sobre todos os 2^k subconjuntos S de $\{1, \dots, k\}$.

6.2 Teorema Central: Grau Algébrico e Colapso do Diferencial

Teorema 6.2 (Colapso do diferencial de ordem d). Seja $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ uma função booleana de grau algébrico exato d (i.e., f possui pelo menos um monômio de grau d em sua Forma Normal Algébrica (ANF), mas nenhum de grau $> d$). Então:

$$D_{\alpha_1, \dots, \alpha_d}^d f(x) = \text{const} \in \mathbb{F}_2 \quad \forall x \in \mathbb{F}_2^n,$$

e $D^{d-1}f$ não é constante em geral.

Prova por indução no grau. Base ($d = 1$): $f(x) = \langle a, x \rangle \oplus c$ (linear). $D_\alpha^1 f(x) = f(x \oplus \alpha) \oplus f(x) = \langle a, \alpha \rangle$, constante. ✓

Passo: Seja $f = m + g$ onde $m = x_{i_1} \cdots x_{i_d}$ é um monômio de grau d e g tem grau $< d$. Por hipótese de indução, $D^d g \equiv 0$ (constante 0, pois g tem grau $< d$ e um diferencial de ordem maior que o grau anula a função). Resta calcular $D^d m$:

$$D_{\alpha_1, \dots, \alpha_d}^d m(x) = \bigoplus_{S \subseteq [d]} \prod_{j=1}^d \left(x_{i_j} \oplus \bigoplus_{s \in S} \alpha_{s, i_j} \right).$$

Expandindo via inclusão-exclusão booleana sobre os 2^d termos, o único coeficiente de grau 0 (independente de x) é o produto das diferenças nos d bits participantes:

$$D_{\alpha_1, \dots, \alpha_d}^d m(x) = \prod_{j=1}^d \left(\bigoplus_{i: \alpha_i \text{ contribui ao bit } i_j} \alpha_{i, i_j} \right) = \text{const.}$$

Todos os termos que contêm variáveis x_{i_j} cancelam em pares. Portanto $D^d f = D^d m \oplus D^d g = \text{const} \oplus 0 = \text{const}$. $\square$

6.3 Aplicação à Função Filtro f (Grau 2)

Corolário 6.3 (Diferencial de 2^a ordem da função filtro). Para $f(x) = s_{20} \oplus (s_0 \wedge s_{10})$ e $\alpha = x^0$, $\beta = x^{10}$ (base canônica de $\mathbb{F}_{2^e}$):

$$D_{\alpha,\beta}^2 f(x) = f(x) \oplus f(x \oplus \alpha) \oplus f(x \oplus \beta) \oplus f(x \oplus \alpha \oplus \beta) = (a_0 \wedge b_{10}) \oplus (a_{10} \wedge b_0),$$

onde $a_k = s_k(\alpha)$, $b_k = s_k(\beta)$.

Proof. Denotando $\delta_k = s_k(\Delta)$ para cada diferença Δ , o Corolário 5.2 nos dá:

$$\begin{aligned} D_{\alpha,\beta}^2 f(x) &= \Delta_\alpha f(x) \oplus \Delta_\alpha f(x \oplus \beta) \\ &= \left[a_{20} \oplus s_0 a_{10} \oplus a_0 s_{10} \oplus a_0 a_{10} \right] \\ &\quad \oplus \left[a_{20} \oplus (s_0 \oplus b_0) a_{10} \oplus a_0 (s_{10} \oplus b_{10}) \oplus a_0 a_{10} \right] \\ &= (b_0 a_{10}) \oplus (a_0 b_{10}), \end{aligned}$$

pois os termos com s_0, s_{10} e a_{20} e $a_0 a_{10}$ cancelam. Para $\alpha = x^0$ temos $a_0 = 1, a_{10} = 0$; para $\beta = x^{10}$ temos $b_0 = 0, b_{10} = 1$; logo:

$$D^2 = (0 \wedge 0) \oplus (1 \wedge 1) = 0 \oplus 1 = 1 = \text{const.} \quad \square$$

A constância do diferencial de 2^a ordem, confirmada empiricamente em `higher_order_differential` com taxa 1.0 em todas as 8 configurações, prova que f tem grau exato 2.

7 Módulo 5: multi_difference_attack

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4.1.3): “by disregarding the output difference when there is a difference in s_{10} , s_{23} , s_{32} , or s_{42} , almost the entire register can easily be recovered.” No projeto, a mesma ideia é generalizada: múltiplas diferenças iniciais $\Delta^{(i)}$ geram características complementares que permitem recuperar bits específicos do estado.

7.1 Recuperação de Bits por Características Complementares

A partir do Corolário 5.2:

$$\Delta z_t = \delta_{20} \oplus (s_0 \wedge \delta_{10}) \oplus (\delta_0 \wedge s_{10}) \oplus (\delta_0 \wedge \delta_{10}).$$

Dado Δ conhecido, o atacante pode isolar cada bit:

Proposição 7.1 (Isolamento de s_0 e s_{10}). (a) Se $\delta_0 = 0$ e $\delta_{10} = 1$, então $\Delta z_t = \delta_{20} \oplus s_0(x_t)$, logo

$$s_0(x_t) = \Delta z_t \oplus \delta_{20}.$$

(b) Se $\delta_0 = 1$ e $\delta_{10} = 0$, então $\Delta z_t = \delta_{20} \oplus s_{10}(x_t)$, logo

$$s_{10}(x_t) = \Delta z_t \oplus \delta_{20}.$$

Proof. Substituição direta na fórmula do Corolário 5.2.

(a): $\delta_0 = 0, \delta_{10} = 1 \Rightarrow \Delta z = \delta_{20} \oplus (s_0 \wedge 1) \oplus (0 \wedge s_{10}) \oplus (0 \wedge 1) = \delta_{20} \oplus s_0$.

(b): $\delta_0 = 1, \delta_{10} = 0 \Rightarrow \Delta z = \delta_{20} \oplus (s_0 \wedge 0) \oplus (1 \wedge s_{10}) \oplus (1 \wedge 0) = \delta_{20} \oplus s_{10}$. $\square$

7.2 Conjunto de Diferenças Utilizadas

O código usa $\Delta_0^{(i)} \in \{x^0, x^1, x^2, x^5, x^{10}\}$ e propaga cada diferença via $\Delta_t^{(i)} = a^t \Delta_0^{(i)}$. Para cada clock t , verificam-se quais diferenças satisfazem as condições da Proposição 7.1, extraíndo $s_0(x_t)$ e $s_{10}(x_t)$.

Teorema 7.2 (Equação diferencial geral: verificação). *O código verifica a equação completa:*

$$\Delta z_t \stackrel{?}{=} \delta_{20} \oplus (s_0^* \wedge \delta_{10}) \oplus (\delta_0 \wedge s_{10}^*) \oplus (\delta_0 \wedge \delta_{10}),$$

usando os gabaritos reais $s_0^* = s_0(x_t)$ e $s_{10}^* = s_{10}(x_t)$. A taxa de concordância é 1.0 em todas as configurações (prova empírica da corretude do Corolário 5.2).

8 Módulo 6: exhaustive_speedup

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4.1.2, *Improving Exhaustive Key Search of Toyocrypt*): “the key $K \oplus s^{126}$ can also be automatically discarded without any consideration, reducing the expected time complexity of an exhaustive key search by a factor of 2^{126} trials. ... even though the exhaustive key search attack does not require the related-key model, it can still benefit from the existence of differentials.” O projeto implementa exatamente este speedup com $\Delta = 1$.

8.1 Propriedade de Simetria dos Pares

Teorema 8.1 (Speedup por complementação via diferencial). *Seja $\Delta = 1 \in \mathbb{F}_{2^e}$ com probabilidade-1 característica diferencial $\Delta_0 \rightarrow \Delta_t = a^t \Delta_0$. Para todo estado inicial $K \in \mathbb{F}_{2^e}$, os fluxos gerados por K e $K' = K \oplus \Delta$ satisfazem:*

$$\{Z(K), Z(K')\} = \{Z(K'), Z(K)\}$$

no sentido de que os dois fluxos formam um par identificável sem testar K' de forma independente. A busca exaustiva sobre 2^e candidatos reduz-se a 2^{e-1} testes, um fator 2 de aceleração.

Proof. Dados os fluxos observados $(Z_{\text{obs}}, Z'_{\text{obs}})$, um candidato K satisfaz a hipótese se e somente se um dos dois ordenamentos coincide:

$$[Z(K) = Z_{\text{obs}} \wedge Z(K') = Z'_{\text{obs}}] \text{ OU } [Z(K) = Z'_{\text{obs}} \wedge Z(K') = Z_{\text{obs}}].$$

Em ambos os casos, ao encontrar o representante par $K^* = K \wedge \bar{1}$ (bit 0 zerado), descarta-se automaticamente $K^* \oplus 1$. Portanto o espaço efetivo de busca é o conjunto de representantes $\{K \in \mathbb{F}_{2^e} : s_0(K) = 0\}$, que tem cardinalidade 2^{e-1} . $\square$

8.2 Verificação Experimental

No código, o speedup teórico de $2\times$ é verificado em $\mathbb{F}_{2^{31}}$ com espaço de 2^{12} candidatos: a busca otimizada testa apenas os candidatos pares ($\Delta = 1$ separa par/ímpar), reduzindo exatamente para 2^{11} testes. O speedup observado é $2.0\times$ de forma consistente.

9 Módulo 7: higher_order_stress e higher_order_scaling

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (§4, parágrafo sobre monômio de grau 3): “if there is a difference in only one bit which is combined through a monomial of degree 3, there is a probability of $1/4$ that there is a difference in the key streams in the corresponding bit.” Para monômios de grau d , a probabilidade cai para $2^{-(d-1)}$, tornando o ataque de 1ª ordem inviável. O diferencial de ordem d restaura a certeza (probabilidade 1), ao custo de 2^d consultas por clock.

9.1 Custo Assintótico do Ataque de Ordem k

Teorema 9.1 (Custo mínimo para quebrar grau d). *Para uma função filtro f de grau algébrico exato d , o diferencial de ordem mínima que produz uma saída constante (e portanto explorável como distinguidor ou para recuperação de estado) é $k = d$, exigindo exatamente 2^d avaliações de f por clock.*

Proof. Segue diretamente do Teorema 6.2: $D^d f = \text{const}$ e $D^{d-1} f \neq \text{const}$. A definição de D^k envolve 2^k pontos (subconjuntos de $\{\alpha_1, \dots, \alpha_k\}$), logo o custo por clock é 2^k . Para $k < d$, o diferencial não é constante e não fornece informação utilizável com probabilidade 1. Para $k = d$, o diferencial colapsa a uma constante determinística. $\square$

9.2 Tabela de Viabilidade

Table 2: Custo do diferencial de ordem k para funções de grau $d = k$				
Grau d	Ordem k	Consultas/clock	Tempo prático	Viabilidade
2	2	4	nanosegundos	Trivial
3	3	8	nanosegundos	Trivial
5	5	32	microsegundos	Fácil
10	10	1 024	milissegundos	Factível
15	15	32 768	segundos	Custoso
20	20	1 048 576	horas	Limite prático
30	30	$\sim 10^9$	anos	Inviável
64	64	$\sim 10^{19}$	$\gg$ idade do universo	Seguro

9.3 Análise do higher_order_stress

O módulo compara:

- **[A]** f_{orig} (grau 2): $D^2 f_{\text{orig}} = \text{const} \Rightarrow$ ataque efetivo. Taxa observada: 1.0.
- **[B]** f_{hard} (grau $N - 1$, produto AND de $N - 1$ bits): $D^2 f_{\text{hard}} \neq \text{const} \Rightarrow$ ataque de 2ª ordem **falha**. Taxa observada: < 1.0 .

Proposição 9.2 (Função hard: resistência ao D^2). *A função $f_{\text{hard}}(x) = \bigwedge_{i=0}^{N-2} s_i(x)$ tem grau algébrico $N - 1$. Portanto $D^2 f_{\text{hard}}$ não é constante para deltas genéricos, e o ataque de 2^{a} ordem não a quebra.*

Proof. A ANF de f_{hard} contém o monômio $s_0 s_1 \cdots s_{N-2}$ de grau $N - 1 > 2$. Pelo Teorema 6.2, $D^2 f_{\text{hard}}$ ainda contém termos dependentes de x de grau $N - 3 > 0$, logo não é constante. $\square$

9.4 Análise do higher_order_scaling

O módulo implementa a função genérica de grau G :

$$f_G(x) = \bigwedge_{i=0}^{G-1} s_i(x) \oplus \bigwedge_{i=0}^{G-2} s_i(x) \oplus \cdots \oplus s_0(x) \oplus s_{G-1}(x),$$

que é projetada para ter grau algébrico *exato* G . O módulo varre ordens $k = 2, 3, \dots$ até encontrar o menor k tal que $D^k f_G$ é constante (nula) em todos os t testados, confirmando $k = G$.

Teorema 9.3 (Verificação empírica do limiar de grau). *Para f_G com grau exato G , o módulo **higher_order_scaling** detecta $k_{\min} = G$ em todas as configurações testadas ($G \in \{2, 3, 4, 5, 8, 10, 15, 20\}$), o que constitui evidência empírica do Teorema 6.2.*

10 Extensão para Funções Filtro de Grau Maior

Referência ao Paper (Biham & Dunkelman)

Biham & Dunkelman (Sumário, §7): “Stream ciphers with high probability differentials are susceptible to many attacks: distinguishing and key recovery in the related-key/IV model, faster exhaustive key searches, and fault analysis.” Para cifras com funções filtro de alto grau, os diferenciais de ordem d ainda se aplicam, porém o custo 2^d pode tornar o ataque impraticável.

10.1 Generalização do Ataque de Recuperação de Chave

Para uma função filtro de grau d , o Corolário 6.3 generaliza-se: $D^d f$ produz uma constante que depende apenas das diferenças $\alpha_1, \dots, \alpha_d$. Seja:

$$C(\alpha_1, \dots, \alpha_d) = D^d f(x) \quad (\text{independente de } x).$$

Escolhendo d diferenças tais que $C \neq 0$ (existência garantida pois o grau é exato d), o atacante forma um *distinguidor de ordem d* : observa 2^d streams correlacionados $\{Z(x \oplus \bigoplus_{i \in S} \alpha_i)\}_{S \subseteq [d]}$ e verifica se seu XOR é C .

10.2 Generalização da Recuperação de Estado

Para $d = 2$, o sistema $M\vec{S}_0 = B$ (Seção 3) pode ser construído porque $\Delta^1 f$ é linear em x_t . Para $d > 2$:

- O diferencial de 1ª ordem $\Delta^1 f$ tem grau $d - 1 > 1$, portanto não é linearizável diretamente.
- O diferencial de ordem $d - 1$ produz expressões lineares em x_t , que podem ser usadas para construir o sistema análogo.
- Especificamente, $D_{\alpha_1, \dots, \alpha_{d-1}}^{d-1} f(x)$ é afim em x (grau 1), possibilitando montar uma equação linear por clock, com custo 2^{d-1} consultas.
- O sistema tem e equações (uma por clock) e e incógnitas (bits de x_0), sendo resolvido por inversão de matriz em $\mathbb{F}_2^{e \times e}$ com $O(e^3)$ operações de bit — idêntico ao caso $d = 2$, mas com custo de coleta 2^{d-1} vezes maior.

10.3 Limiares de Segurança

Definição 10.1 (Grau de segurança de uma função filtro). Uma função filtro f de grau d é considerada *computacionalmente segura* contra ataques diferenciais de ordem d quando:

$$2^d > T_{\text{máx}},$$

onde $T_{\text{máx}}$ é o limiar de consultas tolerado pelo atacante por clock de observação.

Para segurança de 128 bits (padrão moderno), é necessário $d \geq 128$, o que implica $2^{128} \approx 3.4 \times 10^{38}$ consultas por clock — completamente inviável.

Limitação de Segurança

A função filtro $f(x) = s_{20} \oplus (s_0 \wedge s_{10})$ tem grau algébrico $d = 2$, exigindo apenas $2^2 = 4$ consultas por clock para o ataque de ordem 2. Isso a torna **completamente insegura** em qualquer instância do projeto, independentemente do tamanho do campo $\mathbb{F}_{2^e}$.

11 Síntese dos Resultados Experimentais

Table 3: Resultados esperados e propriedades matemáticas por módulo

Módulo	Métrica	Resultado	Garantia Matemática
batch_attack	Acertos / e	100% em todas configs	Thm. 3.1: $\Delta z_t = s_{10}(x_t)$ c/ prob. 1
recover_key	Chave recuperada	Coincide com semente	Thm. 3.4: M invertível sobre $\mathbb{F}_2$
diff_corr	$\phi(\Delta z, s_{10})$	1.0	Thm. 4.2: correlação perfeita
dist_attack	Taxa de discordância	0% (alvo), 50% (rand.)	Thm. 5.1: $\delta_0 = \delta_{10} = 0 \Rightarrow \Delta z = \delta_{20}$
higher_ord.	Taxa $D^2 = \text{pred}$	1.0	Cor. 6.3: $D^2 f = \text{const}$ para grau 2
multi_diff	Equações corretas	100%	Cor. 5.2: equação diferencial completa
exh_speedup	Fator de aceleração	2.0×	Thm. 8.1: pares $\{K, K \oplus \Delta\}$ eliminados
stress_test	D^2 p/ grau- $(N-1)$	Inconsistente	Thm. 6.2: $D^{d-1} f \neq \text{const}$
scaling	$k_{\min}$ para grau G	$k_{\min} = G$	Thm. 9.3: limiar exato do colapso

12 Conclusões

Este relatório demonstrou matematicamente que o projeto implementa de forma rigorosa e completa o framework de criptanálise diferencial de Biham & Dunkelman aplicado a cifras de fluxo baseadas em LCG sobre $\mathbb{F}_{2^e}$.

Os resultados centrais provados são:

- (i) **Propagação determinística (Thm. 2.1):** $\Delta_t = a^t \Delta_0$ — a linearidade do LCG garante que o atacante conhece a diferença interna em qualquer instante.
- (ii) **Extração de bit por diferença (Thm. 3.1):** $f(x) \oplus f(x \oplus 1) = s_{10}(x)$ — a função filtro de grau 2 lineariza-se sob diferença no bit 0.
- (iii) **Recuperação de chave por sistema linear (Thm. 3.4):** a matriz $M \in \mathbb{F}_2^{e \times e}$ é invertível, e $x_0 = M^{-1}B$.
- (iv) **Colapso do diferencial de ordem d (Thm. 6.2):** $D^d f = \text{const}$ é a condição necessária e suficiente para a exploração, exigindo 2^d consultas por clock.

- (v) **Inviabilidade para grau alto (Thm. 9.1):** para $d \geq 30$, o custo $2^d > 10^9$ consultas/clock torna o ataque impraticável; para $d \geq 64$, é computacionalmente seguro.

A vulnerabilidade fundamental da cifra reside no **baixo grau algébrico** ($d = 2$) **da função filtro**, que permite todos os ataques descritos com custo trivial (4 consultas/clock). Aumentar d para ≥ 30 eliminaria os ataques diferenciais práticos, mas introduziria outras vulnerabilidades (não-linearidade insuficiente, correlação com sub-funções, etc.) que requerem análise adicional.

A Identidades de Álgebra Booleana Utilizadas

As seguintes identidades sobre $(\mathbb{F}_2, \oplus, \wedge)$ são usadas nas provas:

$a \oplus a = 0$	(A1: Complemento)
$a \oplus 0 = a$	(A2: Identidade XOR)
$a \wedge (b \oplus c) = (a \wedge b) \oplus (a \wedge c)$	(A3: Distributividade)
$a \wedge 0 = 0, \quad a \wedge 1 = a$	(A4: Absorção)
$a \oplus b = b \oplus a$	(A5: Comutatividade)
$(a \oplus b) \oplus c = a \oplus (b \oplus c)$	(A6: Associatividade)

B Propriedades de $\mathbb{F}_{2^e}$ Utilizadas

- (GF1) $\mathbb{F}_{2^e} = \mathbb{F}_2[x]/\langle h(x) \rangle$ com h primitivo de grau e .
- (GF2) A multiplicação por $a \in \mathbb{F}_{2^e}$ é um automorfismo $\mathbb{F}_2$ -linear.
- (GF3) O grupo multiplicativo $\mathbb{F}_{2^e}^*$ é cíclico de ordem $2^e - 1$.
- (GF4) A projeção $s_k : \mathbb{F}_{2^e} \rightarrow \mathbb{F}_2$, $s_k(y) = [x^k]y$, é $\mathbb{F}_2$ -linear e não-nula.
- (GF5) A base $\{\xi^0, \xi^1, \dots, \xi^{e-1}\}$ é uma base de $\mathbb{F}_{2^e}$ como $\mathbb{F}_2$ -espaço vetorial.

C Referência: Paper Original

- **Autores:** Eli Biham (Technion, Haifa, Israel) e Orr Dunkelman (KU Leuven, Bélgica).
- **Título:** *Differential Cryptanalysis in Stream Ciphers*.
- **Arquivo:** docs/Differential Cryptanalysis in Stream Ciphers.pdf.
- **Cifras analisadas no paper:** Toyocrypt-HS1, A5/1, RC4, Trivium, LILI-128, LILI-II, Py/PyPy, Helix.

- **Correspondência com o projeto:**

- LCG $x_{t+1} = ax_t + c$ em $\mathbb{F}_{2^e} \leftrightarrow$ LFSR primitivo de e estágios.
- $f(x) = s_{20} \oplus s_0 s_{10} \leftrightarrow$ função combinadora de Toyocrypt-HS1 (simplificada a 3 variáveis).
- $\Delta_0 = 1 \leftrightarrow$ diferença $\Delta S = s_0$ de Biham & Dunkelman (§4.1.3).